

The CAP Theorem Trade-off

Availability (AP)

Search &
View



Consistency (CP)

Booking



Functional Requirements

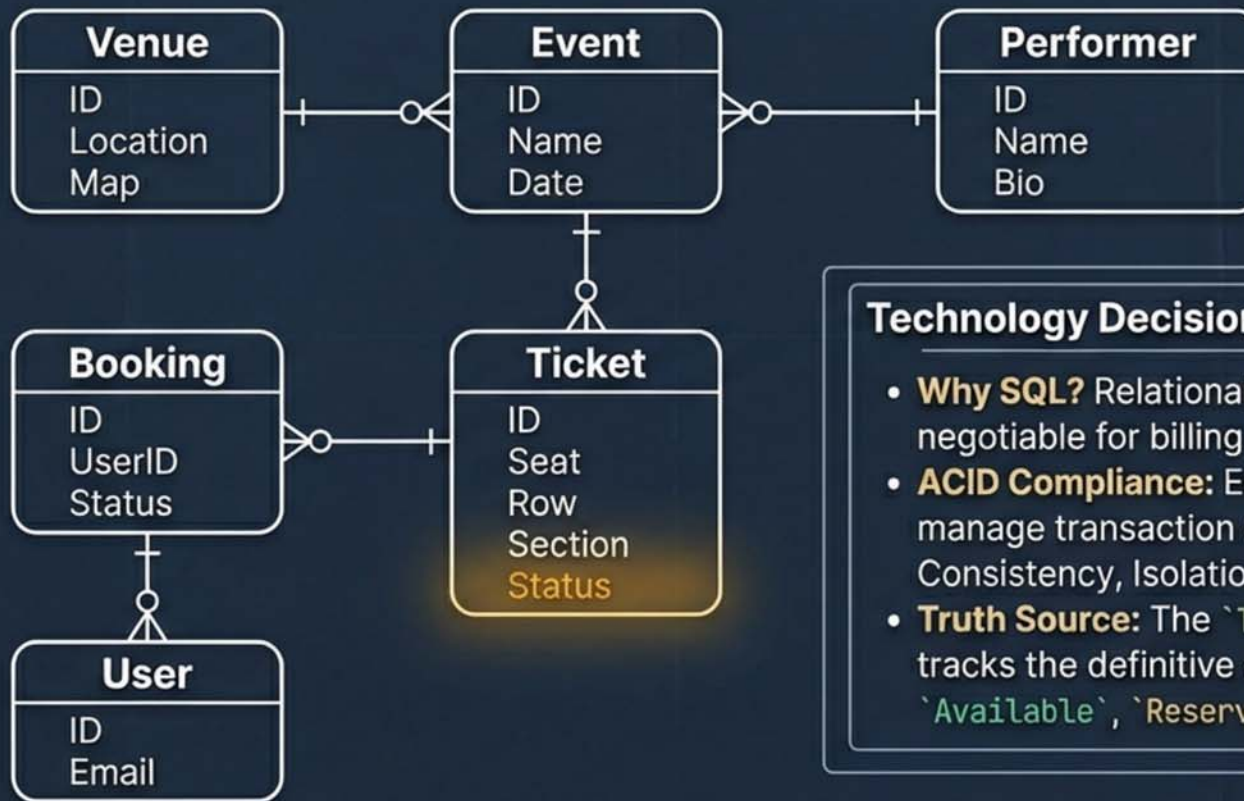
1. **View** Events
2. **Search** Events
3. **Book** Tickets

Non-Functional Requirements

- **Read-Heavy**: 100:1 read-to-write ratio
- **Low Latency**: Search results in <500ms
- **High Throughput**: Support 10 million concurrent users during peak on-sales

The Architectural Split: Search must be **Available**, but Booking must be **Consistent** (No Double Bookings).

The Data Foundation: Entities & Storage



Technology Decision: PostgreSQL

- **Why SQL?** Relational integrity is non-negotiable for billing and inventory.
- **ACID Compliance:** Essential to manage transaction states (Atomicity, Consistency, Isolation, Durability).
- **Truth Source:** The `Ticket` table tracks the definitive status: `'Available'`, `'Reserved'`, or `'Booked'`.

The API Contract

Project Time	
Date	
Revision	

View & Search

```
GET /events/{id}
// Returns Event + Venue +
Performer + Ticket List

GET /search?term={...}
// Returns list of partial
Event objects
```

Phase 1: Reserve

```
POST /bookings/reserve
Body: {
  "ticketIds": ["t1", "t2"]
}
Response: {
  "bookingId": "b_123",
  "expiresAt": "10:10:00"
}
```

Phase 2: Confirm

```
POST /bookings/confirm
Body: {
  "bookingId": "b_123",
  "paymentToken": "tok_visa"
}
// Triggers atomic
transaction
```

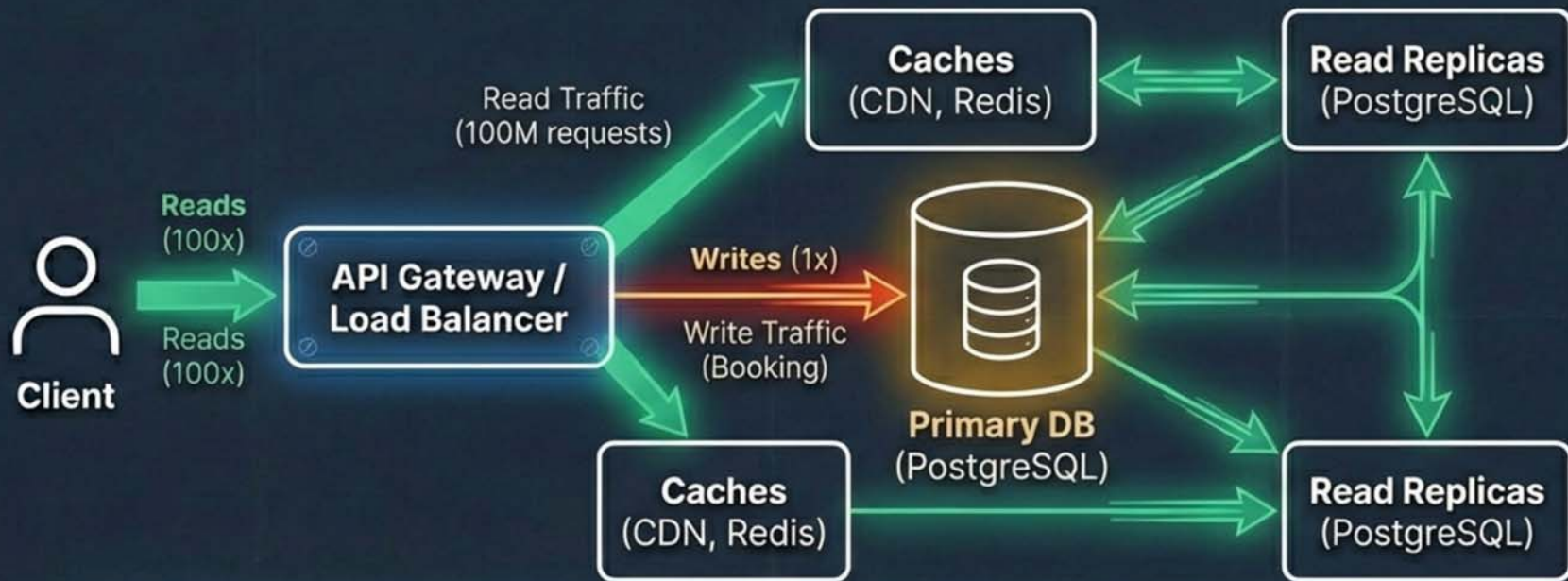
Design Note: Payment processing is integrated into the booking flow to ensure atomic transactions.

High-Level Design (The Strawman)



The Vulnerability: This architecture works for low traffic. However, the database is a single point of failure for 10M concurrent requests, and SQL wildcard search (**LIKE %query%**) is too slow for production scale.

The Scaling Challenge: Read vs. Write



Read Path Strategy:

Optimization Focus: **Latency**

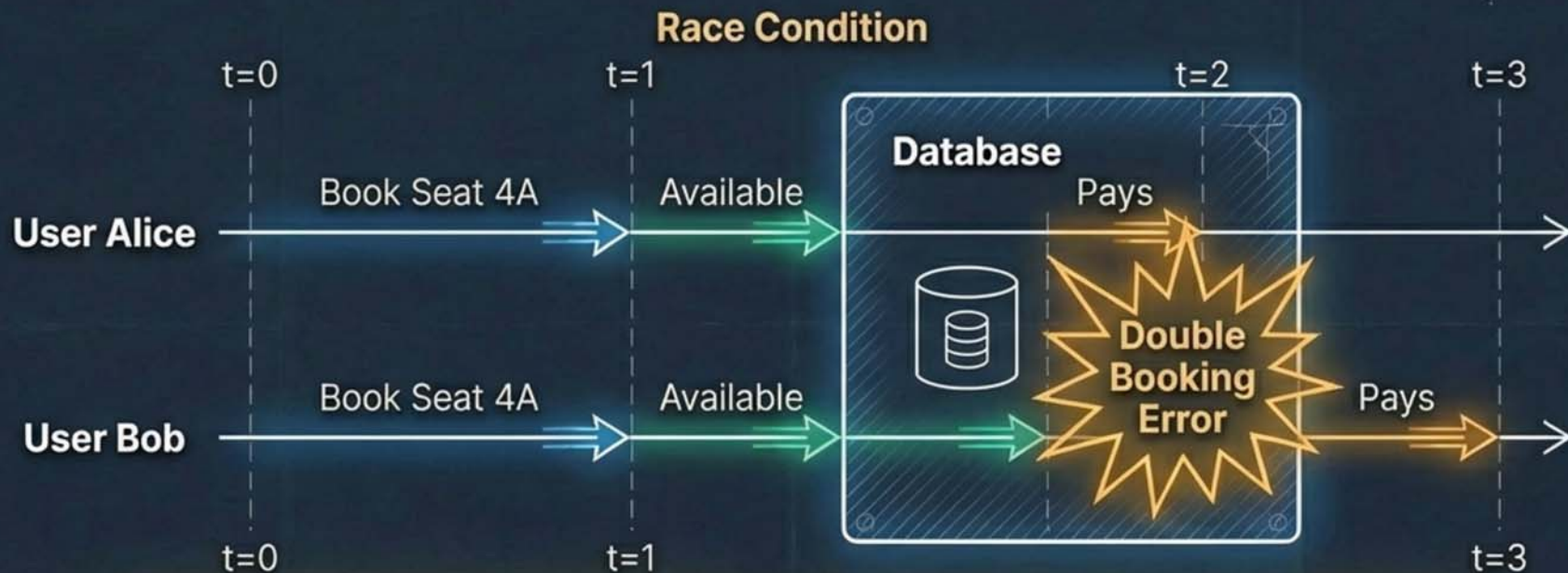
Tactics: Caching, CDNs, Read Replicas.

Write Path Strategy:

Optimization Focus: **Consistency**

Tactics: Locking, ACID Transactions, Queueing.

Deep Dive 1: The 'Double Booking' Nightmare



The Problem:

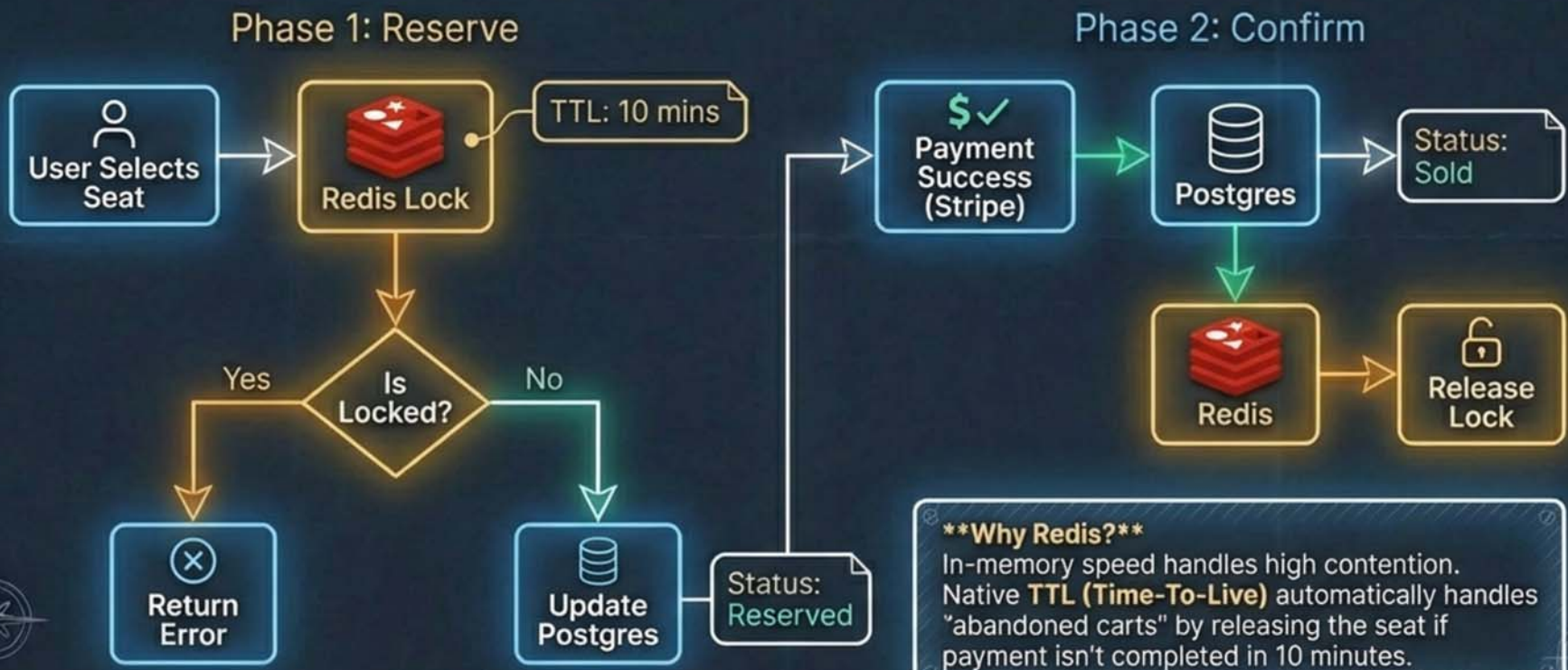
Without locking, parallel requests view stale data.

The Constraint:

We strictly cannot sell the same seat twice.

Solution: Distributed Locking with Redis

Two-Phase Booking



Deep Dive 2: Searching a Haystack

The Problem

```
SELECT *  
WHERE name  
LIKE '%Taylor%'
```



Full table scans are too slow (>2000ms)
for millions of rows.

The Solution: Elasticsearch

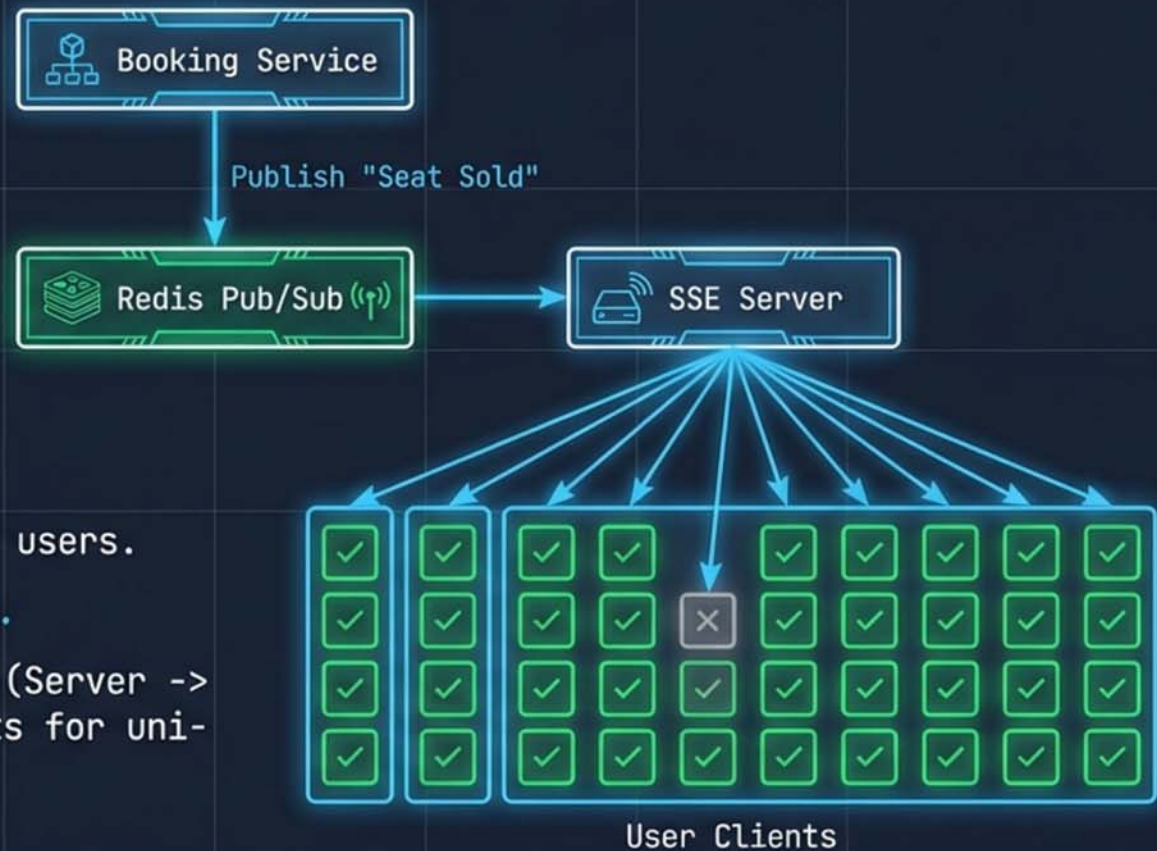
Inverted Index

Word	Doc IDs
Swift	→ [3, 7, 12, 55]
Concert	→ [1, 3, 9]
Oct	→ [2, 3, 7]



Use Inverted Indices for $O(1)$ lookups.
Sync via CDC (Kafka) to decouple search
from the primary DB.

Deep Dive 3: Real-Time Seat Availability

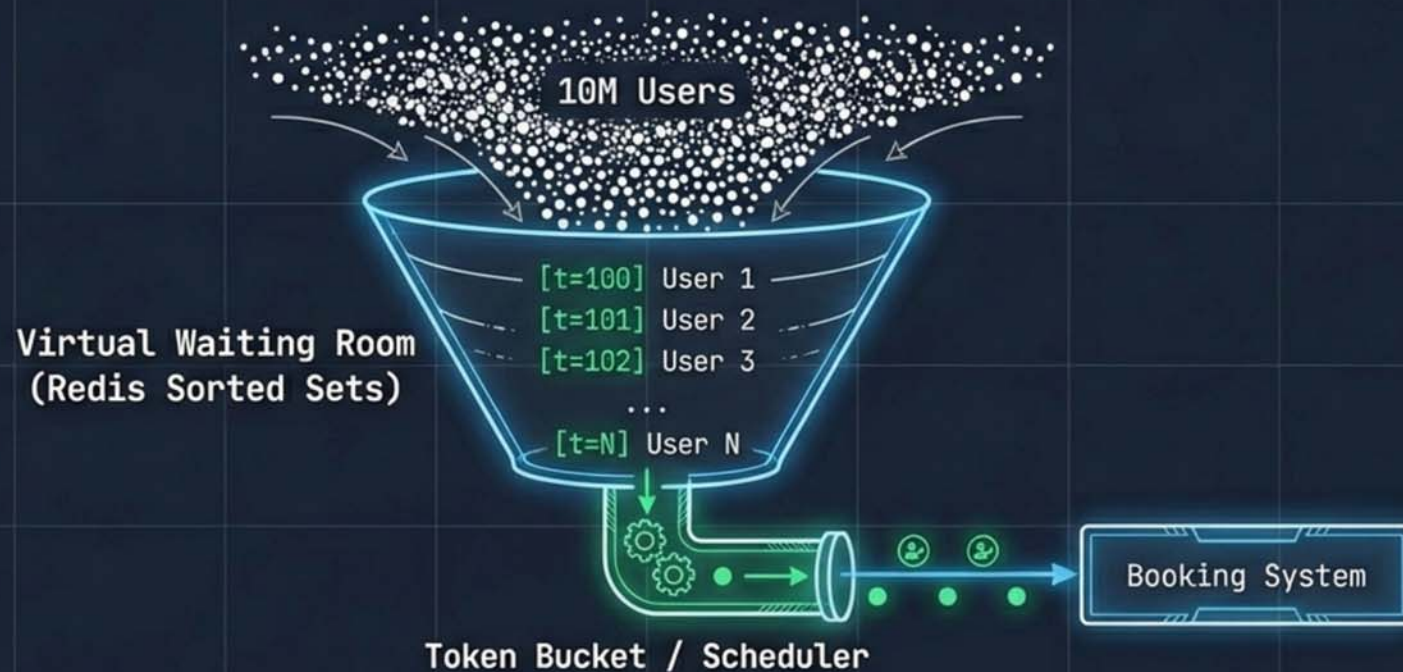


Problem: Stale seat maps frustrate users.

Solution: Server-Sent Events (SSE).

- **Why?** Efficient one-way channel (Server -> Client). Cheaper than Websockets for uni-directional updates.

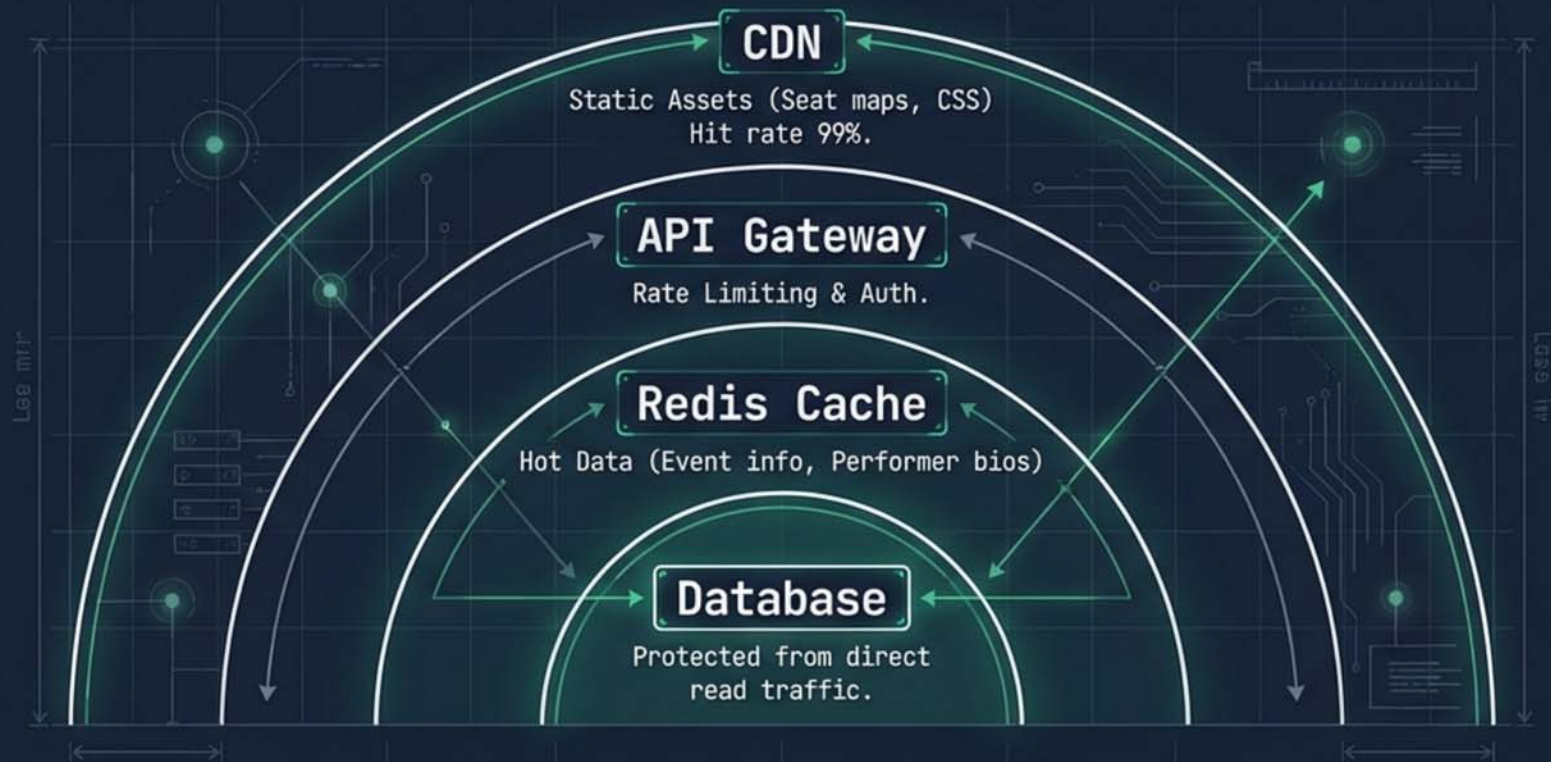
Deep Dive 4: The Traffic Tsunami



Mechanism: Users enter a Redis Sorted Set (**Score = Timestamp**).

The Bouncer: A scheduler allows the top `N` users to enter the booking flow via signed tokens, protecting the database from a thundering herd.

Scaling Reads: Layers of Defense



Strategy reduces read load on the Database by >95%.

The Masterpiece: Final Architecture

